Software Architecture and Design Specification

Project: Event Management System (EMS)

Version: 2.0

Authors: Varun Rao (PES1UG24CS520), Thanishq Raam Patil (PES1UG24CS502), Vishruta (PES1UG24CS538), Nithin Kumar M (PES1UG25CS829)

Date: 10-11-2026

Status: Final (end of Sprint 2)

Course: UE24CS341A Software Engineering, PES University. Group 7 (to confirm)

Revision History

Version	Date	Author	Change Summary
1.0	13-10-2026	Team	Initial architecture and design: component diagram, pattern, security architecture, four sequence diagrams, API design and traceability
1.1	24-10-2026	Team	Review fixes: audit logging of event cancellation, revoked_tokens table, reminder_sent_at flag, traceability corrections
2.0	10-11-2026	Team	Final version after Sprint 2: implementation status, open issues updated, and the implementation deviations with reasons (section 4.6.1)

Approvals

Role	Name	Signature/Date
Course faculty		
Architecture and design	Varun Rao, Thanishq Raam Patil, Vishruta, Nithin Kumar M	10-11-2026

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Event Management System (EMS). It explains how the requirements in the SRS are realized by components, interfaces and interactions, and it is the basis for implementation and for the test plan.

1.2 Scope

The design covers the four functional features of EMS: authentication and role management (F1), event creation and management (F2), event registration and ticketing (F3), and check-in, notifications and reports (F4). Payment processing and native mobile applications are out of scope, as in the SRS.

1.3 Audience

Developers, QA engineers, security reviewers, project guides and evaluators, and the maintenance team.

1.4 Definitions

EMS, SRS, STP, RTM, SAD, REST, JWT, RBAC, STRIDE, TLS, SMTP, ADR, CI. The full list is in the appendix (section 5.1).

2. Document Overview

2.1 How to Use This Document

Section 3 describes the architecture: goals, components, the chosen pattern, the technology stack, risks, traceability and security. Section 4 describes the design: data model, UML sequence diagrams, API design, error handling and UX. Section 5 contains the appendices.

2.2 Related Documents

SRS v1.0 (requirements and security requirements), Software Test Plan v1.0 (test cases and security validation), and the Requirements Traceability Matrix.

3. Architecture

3.1 Goals and Constraints

Goals

- **Correctness under concurrency:** no event is ever overbooked (EVT-NF-004).
- **Security:** protect credentials, enforce roles and ownership, and record security events (EVT-SR-001 to EVT-SR-007).
- **Performance and availability:** 95% of API requests within 2 seconds, and 99% monthly availability (EVT-NF-001, EVT-NF-002).
- **Maintainability:** modular code that each team member can own, with at least 70% unit test coverage (EVT-NF-005).

Constraints

- Technology stack fixed to React.js, Node.js with Express and PostgreSQL.
- No payment processing, so every event is free to register for.
- Delivered by a team of four within the academic semester, using Agile with GitHub, GitHub Actions and SonarQube.
- Email delivery depends on an external SMTP service.

3.2 Stakeholders and Concerns

Stakeholder	Concerns
Attendee	Easy registration, reliable tickets and notifications, privacy of personal data
Organizer	Control of own events, accurate registration and attendance data
Admin	User and category management, platform visibility
Developers	Modularity, clear interfaces, ability to work in parallel
QA engineers	Testable components and traceability to requirements
Security reviewers	Compliance with the security requirements, defence against common attacks
Project guide and evaluators	Fit with the SRS, clear documentation, explainable design

3.3 Component (UML) Diagram

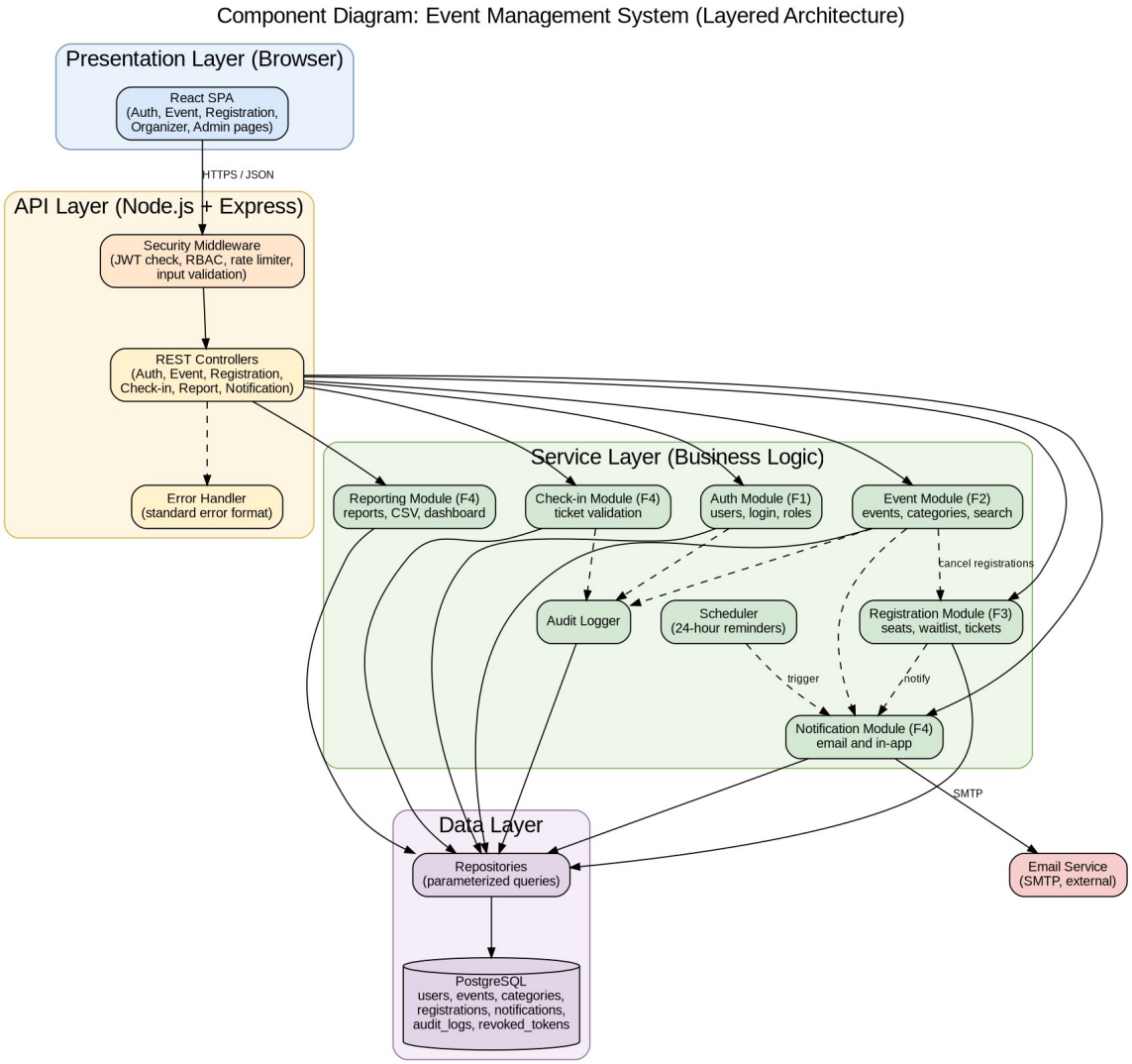


Figure 3.1: Component diagram of EMS. The editable source is `docs/diagrams/component-diagram.dot`.

3.4 Component Descriptions

3.4.1 Frontend and Cross-Cutting Components

Component	Responsibility
React SPA	Single-page web interface with pages for visitors, attendees, organizers and admins. It calls the REST API over HTTPS, keeps the token in memory, and adapts the navigation to the user's role.
Security Middleware	Runs before every protected endpoint. It checks the JWT (EVT-SR-002), enforces role-based access (EVT-F-005), applies rate limiting (EVT-SR-007) and validates and sanitizes input (EVT-SR-003).
REST Controllers	Thin request handlers. They read the request, call one service method and return the response. They contain no business logic.
Error Handler	Converts every error into the standard error response format (see section 4.4).
Audit Logger	Writes audit entries for logins, role changes, event cancellations and check-ins (EVT-SR-006).

Component	Responsibility
Repositories	The only code that talks to the database. All queries are parameterized (EVT-SR-003).
PostgreSQL	Persistent store for users, events, categories, registrations, notifications and audit logs.
Email Service (external)	SMTP provider used by the Notification Module to deliver emails.

The descriptions of the feature modules follow in 3.4.2 (Auth, in `02-f1-authentication-design.md`), 3.4.3 (Event), 3.4.4 (Registration) and 3.4.5 (Check-in, Notification and Reporting).

3.4.2 Auth Module (Component Description)

Item	Description
Purpose	Registers users, authenticates them, enforces account lockout and lets admins manage users and roles
Requirements	EVT-F-001 to EVT-F-007, EVT-SR-001, EVT-SR-002
Main classes	`AuthController`, `AuthService`, `UserService`, `UserRepository`
Depends on	Repositories (users table), Audit Logger, Security Middleware (JWT)
Key rules	Passwords are stored only as bcrypt hashes. Five consecutive failed logins lock the account for 15 minutes. Tokens are signed JWTs that expire after 60 minutes. A deactivated user cannot log in. An admin cannot deactivate their own account. Logins (success and failure) and role changes are written to the audit log. Logout stores the token ID in the `revoked_tokens` table until the token expires.

3.4.3 Event Module (Component Description)

Item	Description
Purpose	Lets organizers create, edit, publish and cancel events, lets admins manage categories, and provides search and filtering of published events
Requirements	EVT-F-010 to EVT-F-016, EVT-NF-002, EVT-NF-006, EVT-SR-003
Main classes	`EventController`, `EventService`, `CategoryService`, `EventRepository`, `CategoryRepository`
Depends on	Repositories (events and categories tables), Registration Module (cancel registrations), Notification Module, Audit Logger
Key rules	New events start as DRAFT and are visible to the public only after publishing. Only the owning organizer can edit, publish or cancel an event. Capacity cannot drop below the number of confirmed registrations. A category that is in use cannot be deleted. Cancelling an event cancels all its registrations and notifies the attendees. Every event cancellation is written to the audit log (EVT-SR-006).

Event statuses: DRAFT, PUBLISHED, CANCELLED.

3.4.4 Registration Module (Component Description)

Item	Description
Purpose	Registers attendees for events, manages the waitlist, handles cancellations with automatic promotion, and provides tickets with QR codes
Requirements	EVT-F-020 to EVT-F-027, EVT-NF-003, EVT-NF-004, EVT-SR-004, EVT-SR-005
Main classes	`RegistrationController`, `RegistrationService`, `TicketService`, `RegistrationRepository`
Depends on	Repositories (registrations and events tables), Notification Module
Key rules	A registration is CONFIRMED when a seat is free and WAITLISTED when the event is full.

Item	Description
	An attendee can have only one active registration per event. Registration and cancellation each run in one transaction with the event row locked, so confirmed registrations never exceed capacity. When a confirmed registration is cancelled, the first waitlisted attendee (first in, first out) is promoted to CONFIRMED in the same transaction and notified. Cancelling after the start time is refused.

Registration statuses: CONFIRMED, WAITLISTED, CANCELLED, CHECKED_IN.

3.4.5 Check-in, Notification and Reporting Modules (Component Description)

Component	Responsibility
Check-in Module (`CheckinController`, `CheckinService`)	Validates a ticket ID for an event and marks the registration CHECKED_IN with a timestamp. Rejects unknown tickets, tickets of another event, and cancelled, waitlisted and already used tickets, each with its own message (EVT-F-030, EVT-F-031). The organizer must own the event.
Notification Module (`NotificationService`, `Mailer`, `Scheduler`)	Creates in-app notifications and sends emails for confirmation, waitlist promotion, cancellation and event cancellation (EVT-F-032). A scheduled job sends one reminder 24 hours before each event to Confirmed attendees only (EVT-F-033). Emails are sent asynchronously with up to 3 retries. Users can list and mark notifications as read (EVT-F-034).
Reporting Module (`ReportController`, `ReportService`)	Builds the event report with counts of Confirmed, Waitlisted, Cancelled and Checked-in registrations and the attendance rate, which is checked-in divided by confirmed as a percentage, and shows 0% when nobody is confirmed (EVT-F-035). Exports the attendee list as CSV (EVT-F-036) and provides the admin dashboard totals (EVT-F-037).
Audit Logger	Records logins, role changes, event cancellations and check-ins with timestamp, user ID and action (EVT-SR-006).

Requirements covered: EVT-F-030 to EVT-F-037, EVT-NF-005, EVT-SR-006, EVT-SR-007.

3.5 Chosen Architecture Pattern and Rationale

Pattern: a layered (n-tier) architecture, deployed as a single backend application with a separate React frontend. The layers are Presentation, API, Service and Data. A layer may only call the layer directly below it.

Rationale

- Separation of concerns: UI, request handling, business rules and data access can be developed and tested separately.
- Parallel work: each team member owns one service module and its controller, repository and tests, which fits the one-feature-per-member rule.
- Simple deployment and debugging for a project of this size, with one Docker container for the backend.

- Transactions are easy: seat allocation and waitlist promotion run in one database transaction in the Registration Module, which prevents overbooking.

Alternatives rejected

- *Microservices:* rejected as too complex for four developers and one semester. They would add network calls, distributed transactions and deployment overhead without a matching benefit.
- *Serverless functions:* rejected because cold starts and scattered state make seat allocation and scheduled reminders harder to control.

3.6 Technology Stack and Data Stores

Concern	Choice
Frontend	React.js with React Router
Backend	Node.js with Express
Database	PostgreSQL, with tables `users`, `events`, `categories`, `registrations`, `notifications`, `audit_logs` and `revoked_tokens` (see section 4.1)
Authentication	JWT, bcrypt for password hashing
Email	Nodemailer with an SMTP provider (a local mail catcher such as MailHog in testing)
Scheduling	node-cron for the 24-hour reminder job
QR codes	`qrcode` library
Testing	Jest and Supertest, with load tests in k6 or JMeter
CI/CD and quality	GitHub Actions, SonarQube, npm audit
Packaging	Docker and Docker Compose

3.7 Risks and Mitigations

ID	Risk	Impact	Mitigation
R1	Two attendees take the last seat at the same moment, leading to overbooking	High	Registration runs in a database transaction with the event row locked, verified by the 500-request concurrency test (TC-NFR-04)
R2	The SMTP service is slow or down	Medium	Emails are sent asynchronously with up to 3 retries, and failures never block the user's request. In-app notifications still work.
R3	A stolen or leaked token is misused	High	Short 60-minute expiry, HTTPS only, logout revokes the token, and the token is kept in memory in the browser
R4	The database is lost or corrupted	High	Docker volume with regular backups, and a seed script to rebuild test data
R5	The 2-second response target is missed as data grows	Medium	Indexes on event and registration lookups, paginated lists, and load tests in CI
R6	Features owned by different members depend on each other (event cancel calls registration and notification)	Medium	Service interfaces are fixed in this document and each cross-module call is covered by an integration test
R7	Delays in the semester schedule	Medium	Agile sprints with story points, with high-priority requirements built first

3.8 Traceability to Requirements

Each requirement of the SRS maps to the component that realizes it, the API endpoint that exposes it and, where one exists, the sequence diagram that shows it. SD-1 is the login diagram, SD-2 the event cancellation diagram, SD-3 the registration diagram and SD-4 the check-in diagram. The test cases for each requirement are listed in the Test Plan (section 13).

Requirement	Component(s)	API endpoint(s)	Sequence diagram
EVT-F-001	Auth Module	POST /api/auth/register	-
EVT-F-002	Auth Module	POST /api/auth/login	SD-1
EVT-F-003	Auth Module, Audit Logger	POST /api/auth/login	SD-1
EVT-F-004	Auth Module, Security Middleware	POST /api/auth/logout	-
EVT-F-005	Security Middleware (RBAC)	All protected endpoints	- (enforced in the middleware before the controller)
EVT-F-006	Auth Module	GET /api/users, PATCH /api/users/:id/role, PATCH /api/users/:id/status	-
EVT-F-007	Auth Module	GET and PUT /api/users/me, PUT /api/users/me/password	-
EVT-F-010	Event Module	POST /api/events	-
EVT-F-011	Event Module, Security Middleware (validation)	POST /api/events, PUT /api/events/:id	-
EVT-F-012	Event Module	PUT /api/events/:id	-
EVT-F-013	Event Module	POST /api/events/:id/publish	-
EVT-F-014	Event Module, Registration Module, Notification Module	POST /api/events/:id/cancel	SD-2
EVT-F-015	Event Module	/api/categories endpoints	-
EVT-F-016	Event Module	GET /api/events, GET /api/events/:id	-
EVT-F-020	Registration Module	POST /api/events/:id/registrations	SD-3
EVT-F-021	Registration Module	POST /api/events/:id/registrations	SD-3
EVT-F-022	Registration Module	POST /api/events/:id/registrations	SD-3
EVT-F-023	Registration Module	POST /api/registrations/:id/cancel	-
EVT-F-024	Registration Module, Notification Module	POST /api/registrations/:id/cancel	- (see 3.4.4)
EVT-F-025	Registration Module	GET /api/registrations/me	-
EVT-F-026	Registration Module (TicketService)	GET /api/registrations/:id/ticket	-
EVT-F-027	Registration Module	GET /api/events/:id/registrations	-
EVT-F-030	Check-in Module	POST /api/events/:id/checkins	SD-4
EVT-F-031	Check-in Module	POST /api/events/:id/checkins	SD-4
EVT-F-032	Notification Module	Internal calls from Event and Registration modules	SD-2, SD-3
EVT-F-033	Notification Module, Scheduler	Internal scheduled job	-
EVT-F-034	Notification Module	GET /api/notifications,	-

Requirement	Component(s)	API endpoint(s)	Sequence diagram
		PATCH /api/notifications/:id/read	
EVT-F-035	Reporting Module	GET /api/events/:id/report	-
EVT-F-036	Reporting Module	GET /api/events/:id/attendees.csv	-
EVT-F-037	Reporting Module	GET /api/admin/dashboard	-
EVT-NF-001	All modules, database indexes	All endpoints	-
EVT-NF-002	Deployment (Docker), health check	GET /health	-
EVT-NF-003	React SPA	-	-
EVT-NF-004	Registration Module (transaction with row lock)	POST /api/events/:id/registrations	SD-3
EVT-NF-005	CI pipeline, test suites, SonarQube	-	-
EVT-NF-006	React SPA	-	-
EVT-SR-001	Auth Module (bcrypt)	POST /api/auth/register, POST /api/auth/login	SD-1
EVT-SR-002	Security Middleware (JWT)	All protected endpoints	SD-1
EVT-SR-003	Security Middleware (validation), Repositories	All endpoints that accept input	-
EVT-SR-004	Deployment (HTTPS reverse proxy)	All endpoints	-
EVT-SR-005	Security Middleware, Registration and Event modules (ownership checks)	Registration and event endpoints	SD-2, SD-4
EVT-SR-006	Audit Logger (used by the Auth, Event and Check-in modules)	Login, role change, event cancel, check-in	SD-1, SD-2, SD-4
EVT-SR-007	Security Middleware (rate limiter)	All endpoints, strictest on login	-

3.9 Security Architecture

Section owner: Thanishq Raam Patil (PES1UG24CS502)

This section shows how the architecture meets the security objectives SO-1 (Confidentiality), SO-2 (Integrity) and SO-3 (Accountability and Availability) and the security requirements EVT-SR-001 to EVT-SR-007 of the SRS.

3.9.1 Trust Boundaries

Boundary	Between	Controls
TB-1	Browser and API	HTTPS with TLS 1.2 or higher, JWT authentication, rate limiting, input validation
TB-2	API and database	Parameterized queries only, a least-privilege database account, credentials in environment variables
TB-3	API and email service	Credentials in environment variables, TLS to the SMTP server, no sensitive data in email bodies

3.9.2 Threat Modeling (STRIDE)

Threat	Example in EMS	Mitigation in the architecture	Requirement
Spoofing	Someone logs in as another user or forges a token	bcrypt password hashes, signed JWTs with a 60-minute expiry, account lockout after 5 failed logins, same error message for wrong password and unknown email	EVT-SR-001, EVT-SR-002, EVT-F-003
Tampering	Modified request data, SQL injection, altered token	Server-side validation and sanitization, parameterized queries, TLS in transit, signature check on every token	EVT-SR-003, EVT-SR-004, EVT-SR-002
Repudiation	A user denies cancelling an event or a role change	Audit log with timestamp, user ID and action for security-relevant actions	EVT-SR-006
Information disclosure	Reading another attendee's ticket, passwords in logs, stack traces in errors	Ownership checks on every object, TLS, hashed passwords, no secrets in logs, generic error messages	EVT-SR-005, EVT-SR-004, EVT-SR-001
Denial of service	Flooding login or search endpoints	Rate limiting per IP (100 requests per minute, 10 login attempts per minute) with HTTP 429, paginated lists, request size limits	EVT-SR-007
Elevation of privilege	An attendee calls an organizer or admin endpoint	Role-based access control in the Security Middleware, object-level ownership checks, role stored on the server side and re-read at login	EVT-F-005, EVT-SR-005

3.9.3 Security Controls by Layer

Layer	Controls
Presentation	The token is kept in memory (not local storage), output is escaped by React, no secrets in the frontend bundle
API	Security Middleware in a fixed order: rate limiter, JWT check, RBAC, input validation. A central error handler returns generic messages.
Service	Ownership and state checks in every service method, and transactions for registration changes
Data	Parameterized queries only, unique constraint on email, and no plaintext secrets in any table
Operations	Secrets only in environment variables, `.env` files git-ignored, `npm audit` and SonarQube in CI, HTTPS enforced by the reverse proxy

3.9.4 Data Protection

Data	Protection
Passwords	Salted bcrypt hash, cost factor 10 or higher, never returned or logged
Personal data (name, email, phone)	Shown only to the owner, to the organizer of an event the person registered for, and to admins
Tokens	Signed, short-lived, never logged
Audit logs	Contain user ID, action and timestamp only, with no passwords or tokens

4. Design

4.1 Design Overview

4.1.1 Design Principles

- **Layered modules:** controllers call services, services call repositories, and no layer skips another.
- **One module per feature:** each feature has its own controller, service, repository and tests, so team members can work in parallel.
- **Thin controllers, rich services:** all business rules (seat limits, ownership, state changes) live in the service layer.
- **Transactions for shared state:** every change to registrations (register, cancel, promote) runs in one database transaction with the event row locked, which guarantees no overbooking (EVT-NF-004).
- **Fail safely:** every error goes through the central error handler, and no internal detail reaches the client.

4.1.2 Code Structure

```
backend/
  src/
    config/          environment and database setup
    middleware/      auth (JWT), rbac, rateLimiter, validate, errorHandler
    modules/
      auth/          controller, service, repository, routes
      events/        controller, service, repository, routes
      registrations/ controller, service, repository, routes
      checkin/       controller, service, routes
      reports/       controller, service, routes
      notifications/ controller, service, mailer, scheduler
    utils/           auditLogger, logger
  tests/             unit and integration tests per module
frontend/
  src/
    pages/           by role: public, attendee, organizer, admin
    components/      shared UI parts
    services/        API client
```

4.1.3 Data Model

Table	Main columns	Notes
`users`	`id`, `name`, `email` (unique), `password_hash`, `phone`, `role` (ATTENDEE, ORGANIZER, ADMIN), `active`, `failed_attempts`, `locked_until`, `created_at`	Used by F1
`categories`	`id`, `name` (unique)	Used by F2
`events`	`id`, `organizer_id` (FK users), `category_id` (FK categories), `title`, `description`, `venue`, `start_time`, `end_time`, `capacity`, `status` (DRAFT, PUBLISHED, CANCELLED), `created_at`	Used by F2
`registrations`	`id`, `event_id` (FK), `user_id` (FK),	`reminder_sent_at` makes sure the 24-

Table	Main columns	Notes
	`status` (CONFIRMED, WAITLISTED, CANCELLED, CHECKED_IN), `ticket_id` (unique), `queue_position`, `created_at`, `checked_in_at`, `reminder_sent_at`	hour reminder is sent only once. Unique on (`event_id`, `user_id`) for active registrations. Used by F3 and F4.
`notifications`	`id`, `user_id` (FK), `type`, `message`, `read`, `created_at`	Used by F4
`audit_logs`	`id`, `user_id`, `action`, `details`, `created_at`	Used by F1, F2 and F4
`revoked_tokens`	`token_id`, `expires_at`	Holds logged-out tokens until they expire (EVT-F-004). Used by F1.

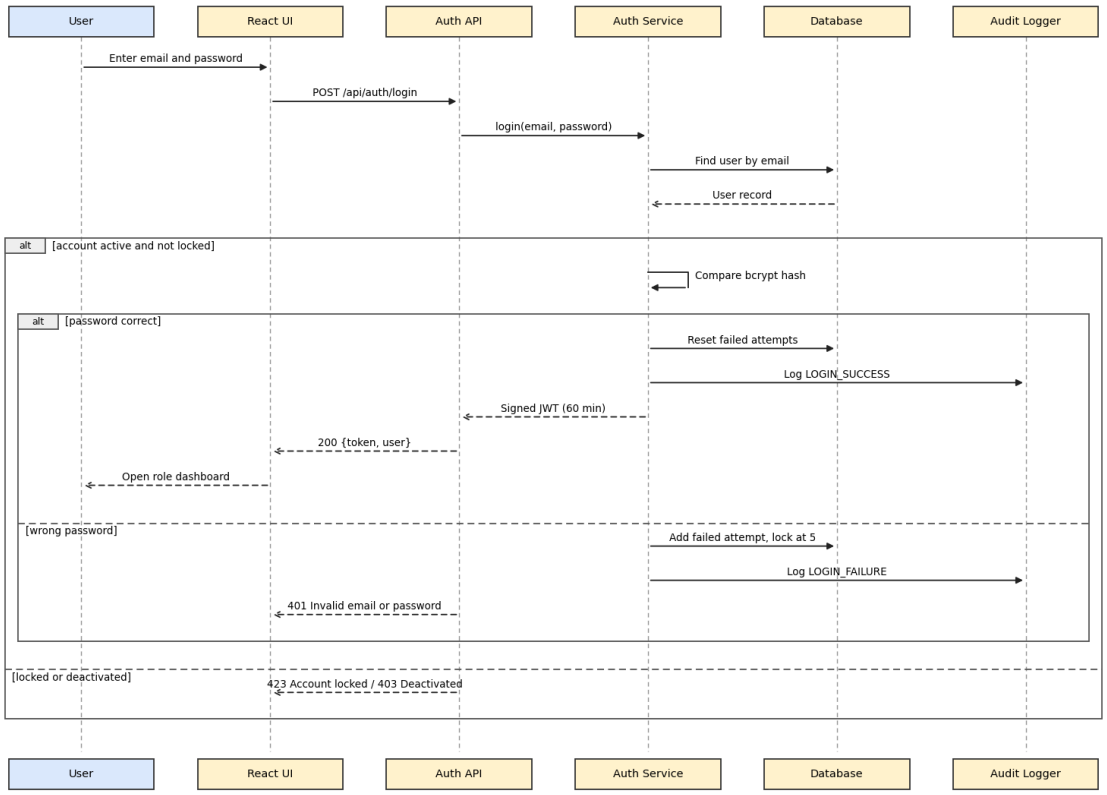
Indexes: `events(status, start_time)`, `events(category_id)`, `registrations(event_id, status)` and `registrations(ticket_id)`, to support the 2-second response target (EVT-NF-001).

4.2 UML Sequence Diagrams

Four sequence diagrams, one for a key flow of each feature. The editable PlantUML sources are in docs/diagrams.

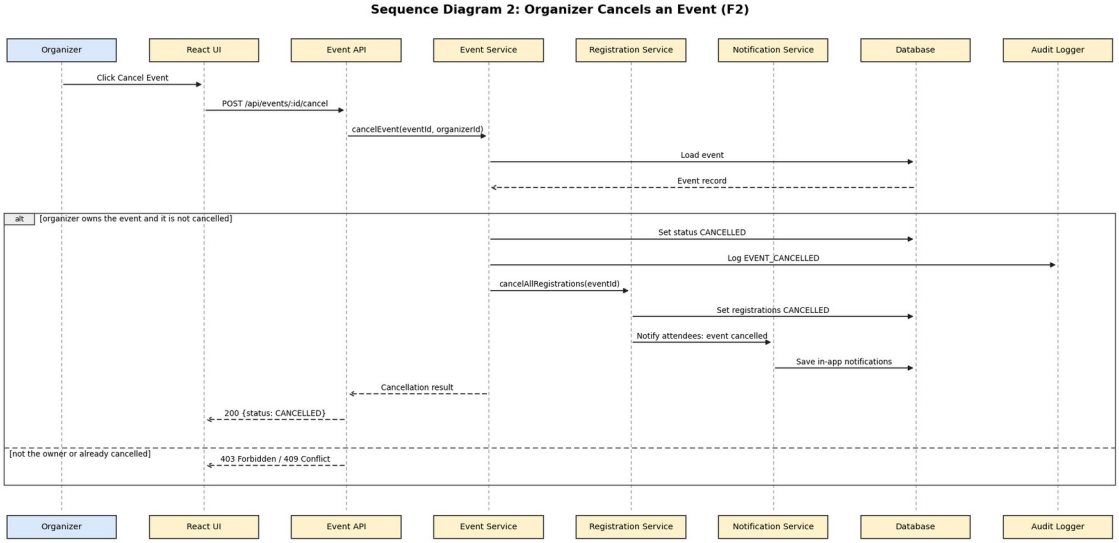
4.2.1 Sequence Diagram: User Login with Account Lockout

Sequence Diagram 1: User Login with Account Lockout (F1)



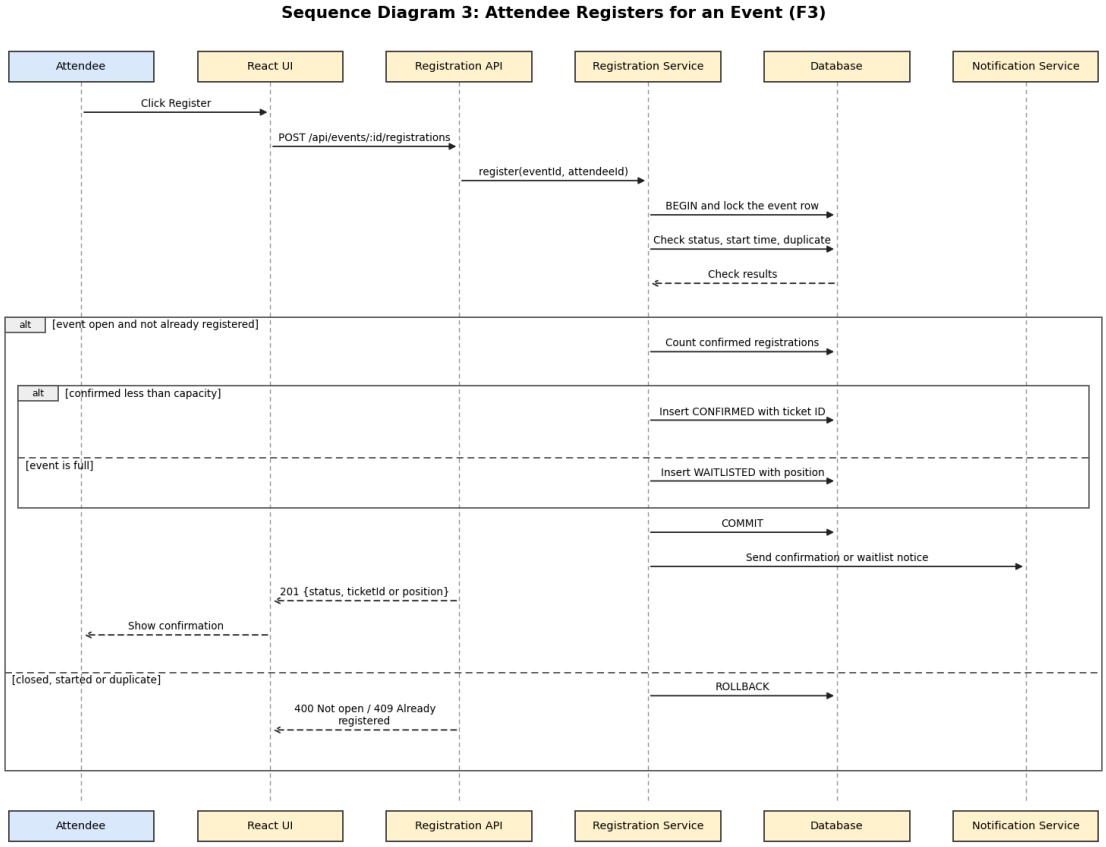
The editable PlantUML source is `docs/diagrams/sequence-1-login.puml`. This flow covers EVT-F-002 and EVT-F-003.

4.2.2 Sequence Diagram: Organizer Cancels an Event



The editable PlantUML source is `docs/diagrams/sequence-2-cancel-event.puml`. This flow covers EVT-F-014, the notification trigger in EVT-F-032 and the audit entry required by EVT-SR-006.

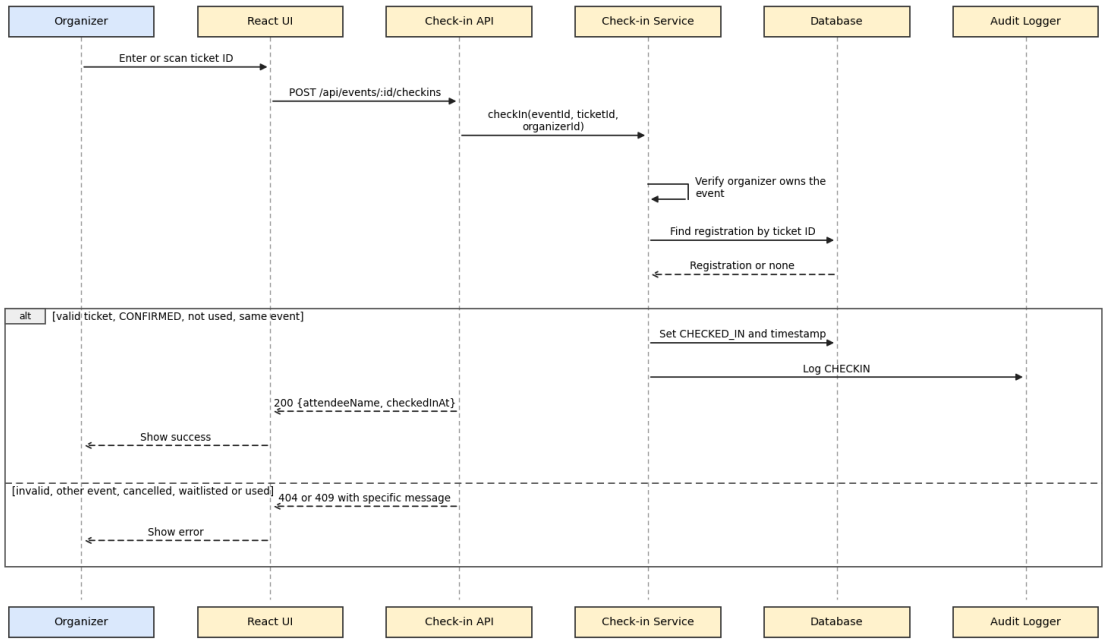
4.2.3 Sequence Diagram: Attendee Registers for an Event



The editable PlantUML source is `docs/diagrams/sequence-3-register-for-event.puml`. This flow covers EVT-F-020, EVT-F-021 and EVT-F-022, and shows how overbooking is prevented (EVT-NF-004).

4.2.4 Sequence Diagram: Organizer Checks In an Attendee

Sequence Diagram 4: Organizer Checks In an Attendee (F4)



The editable PlantUML source is `docs/diagrams/sequence-4-check-in.puml`. This flow covers EVT-F-030 and EVT-F-031.

4.3 API Design

REST API over HTTPS with JSON bodies. All protected endpoints need a valid token (`Authorization: Bearer <token>`); errors use the standard format in section 4.4. Interfaces are listed per component.

4.3.1 API Design: Auth Module

All endpoints use JSON over HTTPS. Errors use the standard format in section 4.4.

POST /api/auth/register

Item	Details
Access	Public
Request	`{ "name": "Asha", "email": "asha@ems.test", "password": "Test@1234", "role": "ATTENDEE" }`
Success	`201 { "userId": 12, "name": "Asha", "email": "asha@ems.test", "role": "ATTENDEE" }`
Errors	`400 VALIDATION_ERROR` (weak password, bad email or role), `409 EMAIL_EXISTS`
Requirement	EVT-F-001

POST /api/auth/login

Item	Details
Access	Public

Item	Details
Request	`{ "email": "asha@ems.test", "password": "Test@1234" }`
Success	`200 { "token": "<jwt>", "expiresIn": 3600, "user": { "id": 12, "name": "Asha", "role": "ATTENDEE" } }`
Errors	`401 INVALID_CREDENTIALS` (same message for wrong password and unknown email), `403 ACCOUNT_DEACTIVATED`, `423 ACCOUNT_LOCKED`, `429 TOO_MANY_REQUESTS`
Requirement	EVT-F-002, EVT-F-003

POST /api/auth/logout

Item	Details
Access	Any logged-in user
Request	Authorization header with the token
Success	`204` with no body. The token is added to a revoked-token list until it expires.
Errors	`401 UNAUTHORIZED`
Requirement	EVT-F-004

User administration and profile

Method and path	Access	Purpose	Main errors	Requirement
`GET /api/users`	Admin	List users with role and status	401, 403	EVT-F-006
`PATCH /api/users/:id/role`	Admin	Body `{ "role": "ORGANIZER" }`, change a user's role	400, 403, 404	EVT-F-006
`PATCH /api/users/:id/status`	Admin	Body `{ "active": false }`, deactivate or reactivate	403, 404, 409 (own account)	EVT-F-006
`GET /api/users/me`	Logged-in user	View own profile	401	EVT-F-007
`PUT /api/users/me`	Logged-in user	Update name and phone number	400, 401	EVT-F-007
`PUT /api/users/me/password`	Logged-in user	Body `{ "currentPassword", "newPassword" }`	400, 401, 403 (wrong current password)	EVT-F-007

4.3.2 API Design: Event Module

POST /api/events

Item	Details
Access	Organizer
Request	`{ "title": "Tech Meetup", "description": "...", "categoryId": 2, "venue": "Hall A", "startTime": "2026-11-05T10:00:00+05:30", "endTime": "2026-11-05T13:00:00+05:30", "capacity": 50 }`
Success	`201 { "eventId": 31, "status": "DRAFT", ... }`
Errors	`400 VALIDATION_ERROR` (empty title, start not in the future, end not after start, capacity

Item	Details
	not 1 to 10000), `401`, `403 FORBIDDEN`
Requirement	EVT-F-010, EVT-F-011

PUT /api/events/:id

Item	Details
Access	Organizer who owns the event
Request	Any editable event fields, using the same validation rules as creation
Success	`200` with the updated event
Errors	`400 VALIDATION_ERROR`, `403 FORBIDDEN` (not the owner), `404 NOT_FOUND`, `409 CAPACITY_BELOW_CONFIRMED`
Requirement	EVT-F-012

POST /api/events/:id/publish and POST /api/events/:id/cancel

Item	Details
Access	Organizer who owns the event
Request	No body
Success	`200 { "eventId": 31, "status": "PUBLISHED" }` or `{ "status": "CANCELLED" }`
Errors	`403 FORBIDDEN`, `404 NOT_FOUND`, `409 INVALID_STATE` (for example publishing a cancelled event)
Requirement	EVT-F-013, EVT-F-014

GET /api/events

Item	Details
Access	Public (published events only)
Query	`q` (keyword), `categoryId`, `from`, `to` (dates), `page` (default 1). The page size is fixed at 10.
Success	`200 { "page": 1, "pageSize": 10, "total": 25, "events": [ ... ] }`
Errors	`400 VALIDATION_ERROR` (bad date or page)
Requirement	EVT-F-016

Other event and category endpoints

Method and path	Access	Purpose	Main errors	Requirement
`GET /api/events/:id`	Public (published) or owner (draft)	Event details	404	EVT-F-016
`GET /api/events/mine`	Organizer	The organizer's own events with registration counts	401, 403	EVT-F-010
`GET /api/categories`	Public	List categories	none	EVT-F-015
`POST /api/categories`	Admin	Add a category	400, 409	EVT-F-015

Method and path	Access	Purpose	Main errors	Requirement
`PUT /api/categories/:id`	Admin	Rename a category	400, 404	EVT-F-015
`DELETE /api/categories/:id`	Admin	Delete a category	404, 409 (in use)	EVT-F-015

4.3.3 API Design: Registration Module

POST /api/events/:id/registrations

Item	Details
Access	Attendee
Request	No body. The attendee comes from the token and the event from the path.
Success	`201 { "registrationId": 88, "status": "CONFIRMED", "ticketId": "TKT-7F3A21" }` or `201 { "registrationId": 89, "status": "WAITLISTED", "waitlistPosition": 1 }`
Errors	`400 EVENT_NOT_OPEN`, `401`, `403 FORBIDDEN`, `404 NOT_FOUND`, `409 ALREADY_REGISTERED`
Requirement	EVT-F-020, EVT-F-021, EVT-F-022

POST /api/registrations/:id/cancel

Item	Details
Access	Attendee who owns the registration
Request	No body
Success	`200 { "registrationId": 88, "status": "CANCELLED", "promotedRegistrationId": 89 }` (the promoted ID is null when nobody was waiting)
Errors	`400 EVENT_NOT_OPEN` (event already started), `403 FORBIDDEN`, `404 NOT_FOUND`, `409 INVALID_STATE` (already cancelled)
Requirement	EVT-F-023, EVT-F-024

Other registration endpoints

Method and path	Access	Purpose	Main errors	Requirement
`GET /api/registrations/me`	Attendee	The attendee's registrations with event name, date and status	401	EVT-F-025
`GET /api/registrations/:id/ticket`	Attendee who owns it	`{ "ticketId", "event", "qrCode" }` where `qrCode` is a data URL of a QR image encoding the ticket ID	403, 404, 409 (not confirmed)	EVT-F-026
`GET /api/events/:id/registrations`	Organizer who owns the event	Registrations with name, email, status and time	403, 404	EVT-F-027

4.3.4 API Design: Check-in and Reporting

POST /api/events/:id/checkins

Item	Details
Access	Organizer who owns the event
Request	`{ "ticketId": "TKT-7F3A21" }`
Success	`200 { "registrationId": 88, "attendeeName": "Asha", "checkedInAt": "2026-11-05T10:12:30+05:30" }`
Errors	`400 VALIDATION_ERROR` (missing ticket ID), `403 FORBIDDEN` (not the organizer of the event), `404 NOT_FOUND` (unknown ticket ID or a ticket of another event), `409 TICKET_ALREADY_USED`, `409 INVALID_STATE` (cancelled or waitlisted ticket)
Requirement	EVT-F-030, EVT-F-031

GET /api/events/:id/report

Item	Details
Access	Organizer who owns the event
Success	`200 { "confirmed": 10, "waitlisted": 2, "cancelled": 1, "checkedIn": 8, "attendanceRate": 80.0 }`
Errors	`401`, `403 FORBIDDEN`, `404 NOT_FOUND`
Requirement	EVT-F-035

Other F4 endpoints

Method and path	Access	Purpose	Main errors	Requirement
`GET /api/events/:id/attendees.csv`	Organizer who owns the event	CSV with name, email, status and check-in time	403, 404	EVT-F-036
`GET /api/notifications`	Logged-in user	The user's in-app notifications	401	EVT-F-034
`PATCH /api/notifications/:id/read`	Owner of the notification	Mark a notification as read	403, 404	EVT-F-034
`GET /api/admin/dashboard`	Admin	Totals of users by role, events by status and registrations	401, 403	EVT-F-037
`GET /health`	Public	Status of the API and the database	503 when the database is unreachable	EVT-NF-002

4.4 Error Handling, Logging and Monitoring

4.4.1 Standard Error Response

Every failed request returns this JSON structure:

```
{
  "error": {
    "code": "EMAIL_EXISTS",
```

```
    "message": "Email already registered",
    "details": []
  }
}
```

`details` carries field-level messages for validation errors, for example `[{ "field": "password", "message": "Must have at least 8 characters" }]`.

4.4.2 HTTP Status and Error Codes

HTTP status	Error code	When it is used
400	`VALIDATION_ERROR`	Invalid or missing input
400	`EVENT_NOT_OPEN`	Registering for a draft, cancelled or started event
401	`UNAUTHORIZED`	Missing, expired, tampered or revoked token
401	`INVALID_CREDENTIALS`	Wrong email or password (one message for both)
403	`FORBIDDEN`	Role not allowed, or the object belongs to someone else
403	`ACCOUNT_DEACTIVATED`	The account was deactivated by an admin
404	`NOT_FOUND`	Object does not exist (also used for unknown ticket IDs)
409	`EMAIL_EXISTS`, `ALREADY_REGISTERED`, `INVALID_STATE`, `TICKET_ALREADY_USED`, `CAPACITY_BELOW_CONFIRMED`	Conflict with the current state of the data
423	`ACCOUNT_LOCKED`	Account locked after 5 failed logins
429	`TOO_MANY_REQUESTS`	Rate limit exceeded
500	`INTERNAL_ERROR`	Unexpected failure. The client gets a generic message and a request ID.

4.4.3 Handling Rules

- Errors are thrown by services as typed errors and converted by the central error handler, so controllers contain no `try/catch` formatting.
- Unexpected errors return HTTP 500 with a generic message. The stack trace is written only to the server log.
- A failed database transaction is rolled back before the error is returned.
- A failed email delivery never fails the user's request. The email is retried up to 3 times and the failure is logged.

4.4.4 Logging

- Structured JSON logs with timestamp, level (`error`, `warn`, `info`), request ID, user ID, route and status code.
- No passwords, tokens or full email bodies are ever logged (EVT-SR-001, EVT-SR-006).
- Security-relevant actions go to the separate audit log (EVT-SR-006).

4.4.5 Monitoring

Item	How
Availability	`GET /health` is checked every minute and returns the status of the API and database (EVT-NF-002)
Response time	The 95th percentile response time is measured in load tests (EVT-NF-001)
Error rate	Count of responses with status 5xx

Item	How
Security signals	Failed logins, locked accounts and HTTP 429 responses taken from the audit log
Email health	Count of failed email deliveries

4.5 UX Design

4.5.1 Principles

- Show only what the user's role needs. The navigation bar changes for visitors, attendees, organizers and admins.
- Keep the main attendee task short: from an event page to a confirmed registration in 3 clicks or fewer (EVT-NF-003).
- Always show the status of a registration (Confirmed, Waitlisted, Cancelled) with text and colour.
- Give clear, specific error messages next to the field that caused them.

4.5.2 Main Screens

Role	Screens
Visitor	Home and event list with search and filters, event details, sign-up, login
Attendee	My registrations, ticket with QR code, notifications, profile
Organizer	My events, event form (create and edit), registration list, check-in screen, event report
Admin	User management, category management, dashboard

4.5.3 Accessibility and Responsiveness

- Layouts work from 360 px to 1920 px wide with no horizontal scrolling.
- Keyboard navigation for all actions, labelled form fields, text alternatives for images and the QR code, and sufficient colour contrast.
- Check-in also works by typing the ticket ID, so a camera is not required.

4.6 Open Issues & Next Steps

Item	Status
Implementation	Complete. All 48 backlog items (30 functional, 6 non-functional, 7 security requirements and 5 setup items) were delivered in two sprints and the development was frozen on 10-11-2026.
Choice of the production SMTP provider	Open. MailHog (mock SMTP) is used for development, testing and the demo; a real provider only needs the SMTP settings changed.
Hosting target for the demo	Decided: Docker Compose on a team laptop, served over HTTPS by nginx at https://localhost:8443.
TLS certificate	A self-signed certificate is created at build time for local use. A public deployment needs a certificate from a certificate authority.
Static analysis	Enforced in CI by ESLint (0 errors) and the 70% coverage threshold. The SonarQube setup is in the repository but the analysis was not run (TC-NFR-05).
Browser support	Tested in Chrome. Firefox and Edge were not tested (TC-NFR-06).
Online payments and paid tickets	Future enhancement, out of scope
Calendar integration and push notifications	Future enhancement
Native mobile applications	Future enhancement

4.6.1 Implementation Deviations

The table lists every place where the implementation differs from version 1.0 of this document or from the SRS, with the reason. They were recorded in the sprint reviews (docs/sprint-reviews) as required for the demo.

#	Sprint	Document	What the document says	What was built	Reason
1	1	SAD 4.2 (F1), EVT-SR-001	Passwords hashed with bcrypt	The `bcryptjs` library is used	Pure JavaScript, so it installs on every team member's Windows machine and in Docker without native build tools. It produces standard bcrypt hashes with cost 10, so EVT-SR-001 is still met.
2	1	SAD 4.1.3, data model	`events.category_id` references `categories`	`category_id` is required (NOT NULL)	Every event must belong to a category for filtering (EVT-F-016), so an event without one is rejected.
3	1	SAD 4.1.3, data model	`audit_logs.user_id`	`user_id` can be empty, and `details` is stored as JSON	A failed login with an unknown email has no user to record, but must still be logged (EVT-SR-006). JSON keeps the details structured.
4	1	SRS EVT-F-032	Emails on confirmation, promotion, cancellation and event cancellation	An email is also sent when an attendee joins the waitlist	Tells the attendee their waitlist position straight away. It is an addition and changes no existing behaviour.
5	1	SRS EVT-F-020, data model	A ticket ID for each confirmed registration	Waitlisted registrations also get a ticket ID when created	The `ticket_id` column is unique and required. The ticket page and check-in (Sprint 2) only accept Confirmed registrations, so a waitlisted ticket cannot be used.
6	1	SRS EVT-SR-007	100 API requests and 10 logins per minute per IP	Same limits, but they can be changed with `API_RATE_LIMIT` and `LOGIN_RATE_LIMIT`	Needed to raise the limit for the load and concurrency tests (EVT-NF-001, EVT-NF-004). The defaults match the SRS.
7	1	SRS EVT-F-016	Browse published events	Only published events that have not started yet are listed	Past events cannot be registered for, so listing them would confuse attendees.
8	2	SAD 2.1 and 3 (Presentation layer), risk R3	The browser keeps the login token in memory, not in local storage	The token is kept in the browser's local storage until it expires	Keeping it only in memory logged users out on every page refresh, which made the app hard to use. The risk is limited by the 60-minute expiry, logout revoking the token on the server, React escaping all output, and the Content-Security-Policy header, which together block the script injection that could read the token.
9	2	SRS EVT-SR-004	HTTPS with TLS 1.2 or higher	HTTPS through nginx at https://localhost:8443, with a self-signed certificate created when the image is built	The project runs on team laptops without a public domain, so no certificate authority can issue a certificate. The browser shows a one-time warning, but the connection is encrypted with TLS 1.2 or 1.3. A real deployment would use a certificate from a certificate authority.
10	2	SRS EVT-F-012, EVT-F-014, EVT-F-027, EVT-F-030, EVT-F-035, EVT-F-036	The organizer of the event can edit, cancel, check in, and see registrations and reports	The same actions are also allowed for admins	Admins need to step in when an organizer is unavailable, for example to cancel an event. Other organizers and attendees are still refused with 403 (EVT-SR-005).
11	2	SRS EVT-F-033	A reminder 24 hours before the start to every Confirmed attendee	The reminder job runs every minute and reminds every Confirmed attendee whose event starts within the next 24 hours, once each	An attendee who registers less than 24 hours before the start still gets a reminder, soon after registering. Each attendee is reminded only once (`reminder_sent_at`).
12	2	SRS EVT-F-035	Report counts of Confirmed and Checked-in registrations and the attendance rate	"Confirmed" in the report counts everyone holding a seat, including those already checked in	A checked-in registration changes status from Confirmed to Checked in. Counting both gives the expected rate (10 confirmed, 8 checked in = 80%).
13	2	SRS EVT-F-006	Admin changes roles and deactivates or reactivates accounts	An admin cannot change their own role or deactivate themselves (409), and reactivating an account also clears an account lock; both actions are audited	Stops the system from being left without an admin by mistake, and gives admins a way to unlock an account early. Status changes are logged as `ACCOUNT_STATUS_CHANGED` in addition to `ROLE_CHANGED` (EVT-SR-006).
14	2	SRS EVT-F-014	Organizer cancels an event	An event can be cancelled or edited only before it starts	Changing an event that has already started would make check-in and reports inconsistent.

#	Sprint	Document	What the document says	What was built	Reason
15	2	SRS EVT-NF-005	70% unit test coverage of backend code	The 70% gate covers the backend `src` code; test scripts (`backend/scripts`) and database scripts are excluded	The load, concurrency and uptime scripts are test tools run by hand, not application code.

5. Appendices

5.1 Glossary

Term	Meaning
ADR	Architecture Decision Record
CI	Continuous Integration
EMS	Event Management System
JWT	JSON Web Token
RBAC	Role-Based Access Control
REST	Representational State Transfer
RTM	Requirements Traceability Matrix
SAD	Software Architecture and Design Specification
SMTP	Simple Mail Transfer Protocol
STRIDE	Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
TLS	Transport Layer Security
Waitlist	First-in-first-out queue of attendees waiting for a seat

5.2 References

IEEE 42010 (architecture description), OWASP Top 10 and OWASP ASVS, NIST SP 800-160, and the project's SRS and Test Plan.

5.3 Tools

Graphviz and draw.io for component diagrams, PlantUML for sequence diagrams, Postman and Swagger for API documentation, GitHub Projects for the backlog, SonarQube for static analysis.